

DeepSeek review of μ OS++ IIIe

Kernel, ports, platforms and tests

DeepSeek review of μ OS++ Ille — kernel, ports, platforms and tests

- Executive summary

μ OS++ Ille SMP kernel — read-only code review

- Scope
- Summary
- Findings
- Good practices

μ OS++ Ille AArch32 / AArch64 port review

- Scope
- Summary
- Findings
- AArch32 vs AArch64 divergences
- Good practices

Code review — μ OS++ Ille Cortex-M and POSIX architecture ports

- Scope
- Summary
- Findings
- Good practices

Read-only code review — μ OS++ Ille test harness / platforms / build system

- Scope
- Summary
- Findings
- Documentation drift
- Good practices

DeepSeek review of μ OS++ Ille — kernel, ports, platforms and tests

A read-only review of the three working copies that make up the μ OS++ Ille test workspace: the **SMP kernel** and its harness (`micro-os-plus-iii-smp.git`), the **architecture ports** (`micro-os-plus-iii-{aarch32,aarch64,cortexm,posix-arch}.git`) and the **test platforms** (`tests/platforms/*` , `tests/sources/*` , `test_smpl/`). Nothing in the reviewed trees was modified; this document and its PDF are the only artefacts.

The review was split into four independent passes — kernel, AArch32/AArch64 ports, Cortex-M/POSIX ports, and the test harness/build system — each reading the source and reporting findings with `file:line` evidence and a severity (Critical / High / Medium / Low / Nit). Each pass is reproduced below under its own part.

Executive summary

The design is deliberate and, in the SMP-sensitive core, well documented: the cross-core lock, the deferred publish/claim protocol, the secondary-core release and the semihosting verdict path are coherent and largely correct. The findings below cluster into four themes.

1. Missing synchronisation on shared scheduler state (kernel, High). `join()` , `kill()` and the idle reaper read another core's `current_thread_[]` , thread `state_` and `context_.port_.stack_ptr` without atomics/acquire, while the ports write `stack_ptr` with release stores — a formal C++ data race (`src/rtos/os-thread.cpp:1104,1460` ; `src/rtos/os-idle.cpp:91`). The semihosting file-descriptor table is likewise unsynchronised.

2. Cortex-M SMP: divergence between the two near-duplicate cores (Critical). `switch_stacks()` 's "no ready thread" path spins forever **while holding the kernel spinlock**, freezing the other core (`os-core-m33.cpp:471` , `os-core-rp2350.cpp:448`); the RP2350 copy lost the M33's PRIMASK save/restore and calls `port_put_lock(0)` (`os-core-rp2350.cpp:352`); and neither masks local interrupts around the thread picker, unlike upstream `os-core.cpp:842-869` . The two cores are ~91% duplicated, which is how the drift arose.

3. Memory and time (kernel/ports, High). `malloc` / `new` memory resources discard the requested alignment and no over-aligned `operator new` is defined (`include/cmsis-plus/estd/malloc.h:242,279`); `block_pool::internal_construct_` has an inverted `std::align` failure assert and then overstates the arena (`src/memory/block-pool.cpp:167-179`). On the Pi the AArch32 `clock_highres` reads the raw, admittedly-wrong `CNTFRQ` instead of the calibrated frequency — high-res durations are ~19× off (`aarch32 .../rtos/os-core.cpp` `os-inlines.h:309,315`); the AArch64 MMU leaves TTBR1 walks enabled with `T1SZ=0` (`aarch64 .../mmu.cpp:139`).

4. Test-harness wiring (tests, High/Medium). The Luckfox Lyra harness never calls `smp_install_boot_threads()` / `start_secondary_cores()` , so its suites silently run on core 0 only

(`tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp:94`); the four Pi platforms define `micro-os-plus::platform` / `platform-support` targets nothing links, and their base interface hard-wires `QEMU_BUILD` and the QEMU linker script (`tests/platforms/aarch32-rpi3b/cmake/platform-library.cmake`); the pinned-toolchain guard exists only in the AArch32/AArch64 definitions.

Also notable. The DWC2 USB gadget stack enumerates only when compiled at `-O0` (a single `-O2` transformation breaks the EP0 path — worked around in `cortexm .../tests.cmake:162`); `SYS_EXIT` 's parameter shape differs between the AArch32 (reason-by-value) and AArch64 (two-word block) ports; secondary-core release timeouts are silent; and the POSIX port's macOS tick is process-directed rather than per-CPU.

Each part below gives the full findings, with confirmed defects separated from suspicions and, where relevant, a note on good practices worth keeping.

µOS++ IIIe SMP kernel — read-only code review

Scope

Reviewed, read-only, under `/home/dan/Work` :

- Kernel sources `micro-os-plus-iii-smp.git/src/` (rtos, memory, semihosting, posix-io, startup, utils, libcpp).
- Kernel headers `micro-os-plus-iii-smp.git/include/cmsis-plus/` (rtos, memory, utils, posix-io, diag).
- Harness test sources `micro-os-plus-iii-smp.git/tests/sources/` and shared runners `micro-os-plus-iii-smp.git/test_smpl/`.

Context used to judge the kernel (not modified): the port contract `port/smp-common/cmsis-plus/rtos/port/os-decls.h` declares the cross-core lock (`_smp_klock` , per-core `lock_state[]` , `_port_ctx_pending[]`); the lock and context-switch implementations live in the architecture repos (`micro-os-plus-iii-aarch64` , `-aarch32` , `-posix-arch`), so a few observations about lock/memory-ordering are marked as depending on those ports. The current working tree was reviewed as-is (it already contains uncommitted condvar/detach changes over commit `55000fe`).

No files were modified except this report.

Summary

The kernel core is in good shape: the interrupt/scheduler RAIL critical sections correctly acquire the recursive per-core kernel lock, the deferred-publish/claim pattern in the ports is documented at length, and the SMP-sensitive paths (`join()` , `kill()` , idle reaper, ready-list picker) already carry fixes for earlier

life-cycle races. The most important remaining risks are **cross-core non-atomic reads of `current_thread_[]`, `state_` and `stack_ptr`** (technically a C++ data race) and **memory resources that ignore the requested alignment**, including the absence of over-aligned `operator new / delete` overloads. Beyond those, most findings are concrete but lower-severity defects in the semihosting syscall shim, the POSIX fd table, the first-fit/block-pool allocators, and the list/iterator helpers, plus shell-runner robustness issues.

Severity tally: 0 Critical, 3 High, 11 Medium, 9 Low/Nit (23 findings).

Findings

[HIGH] Cross-core non-atomic reads of the scheduler's shared state — `src/rtos/os-thread.cpp:1104, 1460`; `src/rtos/os-idle.cpp:91`; `include/cmsis-plus/rtos/os-sched.h:59`

```
// os-thread.cpp
for (unsigned c = 0; c < OS_NCPU; ++c)
    if (scheduler::current_thread_[c] == this) { still_running = true; break; }
...
bool busy = (context_.port_.stack_ptr == nullptr);
for (unsigned c = 0; c < OS_NCPU; ++c)
    if (scheduler::current_thread_[c] == this) { busy = true; ... }
```

`current_thread_` is declared `thread* volatile current_thread_[OS_NCPU]` and `state_` is `state_t volatile` (`os-thread.h:1699`); `context_.port_.stack_ptr` is written by the ports with `__atomic_store_n(..., __ATOMIC_RELEASE)` (see `posix-arch/src/rtos/os-core.cpp:315,354`). `join()`, `kill()` and the idle reaper read all of these from *other* cores without the kernel lock (the lock is dropped for the wait/yield loops) and without atomics or a matching acquire. `volatile` orders only the compiler's access to that object, not CPU visibility or ordering.

Impact: a formal C++ data race (UB) on the hot scheduler path; on weakly-ordered cores the `stack_ptr` / `current_thread_` update can be observed late, extending the window in which `join()` / `kill()` may decide a thread is quiescent while another core is still restoring or about to run it. The busy-wait loops make a wrong decision unlikely in practice, so this is **confirmed UB / suspected rare misbehaviour**, not a demonstrated failure.

Suggestion: make `current_thread_[]`, `state_` and `stack_ptr` `std::atomic` (or use `__atomic_load_n(..., __ATOMIC_ACQUIRE)` in these three readers), or read them under the kernel lock in a short retry loop.

[HIGH] Memory resources ignore the requested alignment — include/cmsis-plus/memory/malloc.h:242, 279; src/libcpp/new.cpp (no aligned overloads)

```
// malloc_memory_resource::do_allocate
// Ignore alignment for now.
void* mem = std::malloc (bytes);
...
// new_delete_memory_resource::do_allocate
// Ignore alignment for now.
void* mem = ::operator new (bytes);
```

`memory_resource::allocate(bytes, alignment)` passes an alignment that both resources discard. `operator new` / `operator delete` in `src/libcpp/new.cpp` also define only the `size_t` and `nothrow` forms — there is no `operator new(std::size_t, std::align_val_t)` / `operator delete(void*, std::align_val_t)`.

Impact (confirmed): any over-aligned type (`alignof(T) > alignof(std::max_align_t)`, e.g. a 64-byte cache-line struct, `alignas(64)` buffers, SIMD types) allocated through these resources receives under-aligned storage → UB and possible faults. On the host it silently succeeds because glibc returns 16-byte alignment, so it will not show up in the POSIX tests.

Suggestion: honour `alignment` (over-allocate + store the offset, or `std::aligned_alloc` / `posix_memalign` on host, aligned `operator new` in C++), and add the `align_val_t` overloads to `new.cpp`.

[HIGH] `block_pool::internal_construct` has an inverted failure assert (and overstates the arena) — src/memory/block-pool.cpp:167

```
res = std::align (alignof (void*), blocks * block_size_bytes_,
                 pool_addr_, align_sz);
// std::align() will fail if it cannot fit the adjusted block size.
if (res != nullptr)           // <-- should be "if (res == nullptr)"
{
    assert (res != nullptr);
}
total_bytes_ = blocks_ * block_size_bytes_;
```

On failure (`res == nullptr`) nothing is reported; on success the assert is trivially true. `first_fit_top::internal_construct` does this correctly (`first-fit-top.cpp:57`, `if (res == nullptr)`), which shows the intended pattern. `total_bytes_` is then set to the un-aligned requested size even if the arena cannot actually hold it.

Impact (confirmed): an undersized/misaligned pool silently advertises more capacity than exists; a later `do_allocate` writes block-link words past the arena. Debug builds will not catch it.

Suggestion: assert on `res == nullptr` and compute `total_bytes_` from the adjusted `align_sz`.

[MEDIUM] Semihosting fd table is unsynchronised and `newslot()` does not reserve — `src/semihosting/c-syscalls-semihosting.cpp:140, 300, 358`

```
static int __semihosting_newslot (void) { ... return i; } // does not mark it used
...
int fd = __semihosting_newslot (); // pick
...
int fh = call_host (SEMIHOSTING_SYS_OPEN, block); // host call
if (fh >= 0) { openfiles[fd].handle = fh; ... } // mark much later
```

`openfiles[]` is a plain global array; `__semihosting_findslot`, `__semihosting_newslot`, `__posix_open` and `__posix_close` all mutate it without any lock. The slot is claimed only after the (slow) host call, so two threads/cores racing `open()` can be handed the same `fd`. The same global table and the single semihosting debug channel are also shared across cores.

Impact (confirmed for single-threaded use; **suspected** under concurrent open/close): two different `io` objects share one descriptor slot, one handle leaks, or a close recycles a slot still in use.

Suggestion: guard the fd table with a critical section (or a dedicated spinlock) and reserve the slot before the host call, releasing it on failure.

[MEDIUM] `__posix_stat` produces an invalid `st_mode` (`S_IFREG` OR `S_IFCHR`) — `src/semihosting/c-syscalls-semihosting.cpp:556, 268`

```
// __posix_stat
buf->st_mode |= S_IFREG | S_IREAD;
int res = __semihosting_stat (fd, buf);
...
// __semihosting_stat
st->st_mode |= S_IFCHR;
```

`S_IFREG` (0x8000) and `S_IFCHR` (0x2000) share the `S_IFMT` mask, so the result is 0xA100 — neither a regular file nor a character device. `S_ISREG()` / `S_ISCHR()` both fail.

Impact (confirmed): code that branches on the file type (e.g. FatFs/posix users, `ls`-like diagnostics) misclassifies every file opened via `stat()`.

Suggestion: assign rather than OR, or make `__semihosting_stat` set `S_IFCHR` only when the caller did not already set a type.

[MEDIUM] `file_descriptors_manager::deallocate` / `socket` dereference a possibly-null slot — `src/posix-io/file-descriptors-manager.cpp:197, 211`

```
descriptors_array__[fildes]->clear_file_descriptor (); // deallocate, no null check
...
auto* const io = descriptors_array__[fildes];           // socket
if (io->get_type () != ...)                             // io may be nullptr
```

`io(fildes)` deliberately returns `nullptr` for an empty slot (line 90), but `deallocate()` and `socket()` index the array directly and dereference. `assign()` (line 154) can also install an `io` over an occupied slot without clearing the previous owner.

Impact (confirmed): a double `close()` / `close()` of a never-opened descriptor, or `getsockopt`-style calls on a stale fd, crash on a null dereference instead of returning `EBADF`. A re-`assign` leaks the previous `io`'s descriptor bookkeeping.

Suggestion: null-check the slot in `deallocate()` / `socket()` and return `EBADF`; in `assign()` reject an already-occupied slot (or clear it first).

[MEDIUM] CMSIS timeout conversion overflows 32 bits before widening — `src/rtos/os-c-wrapper.cpp:3755` (and 3804, 3883, 4067, 4178, 4371, 4661, 4759, 4897, 5060)

```
result_t res = sysclock.sleep_for (
    clock_systick::ticks_cast ((uint64_t)(millisec * 1000u));
```

`millisec` is `uint32_t`; `millisec * 1000u` wraps modulo 2^{32} before the cast to `uint64_t`, for any `millisec > 4_294_967` ms (≈ 71.6 min). `osWaitForever` is special-cased in most callers, but a large finite timeout is not.

Impact (confirmed): timeouts above ~ 71 min silently become a small/zero timeout — the call returns early (`osEventTimeout`) instead of blocking as asked.

Suggestion: `(uint64_t)millisec * 1000u`, or better convert via a helper that takes `uint64_t` throughout.

[MEDIUM] `condition_variable::timed_wait` decides `ETIMEDOUT` from the wall clock after re-locking — `src/rtos/os-condvar.cpp:771, 811`

```
clock::timestamp_t timeout_timestamp = clk->steady_now () + timeout;
...
res = mutex.lock ();
if (res == result::ok && clk->steady_now () >= timeout_timestamp)
    res = ETIMEDOUT;
```

The timeout is inferred from the clock *after* the mutex is re-acquired, not from which event (timer expiry vs `signal` / `broadcast`) actually woke the thread. A thread that was signalled just after its deadline — but that re-acquires the mutex before the timer node handler runs — is reported as `ETIMEDOUT`. Also `steady_now() + timeout` can wrap `timestamp_t` with no check.

Impact (confirmed): spurious `ETIMEDOUT` even though the condition was signalled (POSIX allows either outcome only when the timeout genuinely expired first), and wraparound for huge timeouts. Condition-variable wait loops that treat `ETIMEDOUT` as fatal will misbehave.

Suggestion: remember whether the timeout node's `action()` ran (or whether the CV link was cleared by signal/broadcast) and only report `ETIMEDOUT` in the timer case; saturate the deadline.

[MEDIUM] `int` truncation of a `size_t` difference in the allocators — `src/memory/first-fit-top.cpp:143; src/memory/lifo.cpp:91`

```
int rem = static_cast<int> (chunk->size - alloc_size);
if (rem >= 0) { ... }
```

When `chunk->size < alloc_size` the subtraction underflows to a huge `size_t`; converting that to `int` is implementation-defined (pre-C++20) and only "works" because the low 32 bits happen to be negative on the tested ABIs. On a 64-bit `size_t` this is fragile and non-portable, and it silently relies on two's-complement wrapping.

Impact (confirmed portability/UB risk; correct on GCC/Clang targets): the "does it fit" test can be mis-evaluated by a conforming compiler with different conversion semantics.

Suggestion: compare directly with `chunk->size >= alloc_size` and use `std::size_t rem = chunk->size - alloc_size;`, or cast through `std::ptrdiff_t` after the guard.

[MEDIUM] `thread::detach()` rejects parentless threads and is fragile on SMP — `src/rtos/os-thread.cpp:1004`

```
if (state_ == state::destroyed || parent_ == nullptr)
{
    instrumentation::thread::detach_retval (this, EINVAL);
    return EINVAL;
}
child_links_.unlink ();
scheduler::top_threads_list_.link (*this);
parent_ = nullptr;
```

Top-level threads are created with `parent_ == nullptr` (`internal_construct_`, line 567-575) and appear on `top_threads_list_`; the new code therefore returns `EINVAL` for exactly those threads, even though they are joinable. It also performs the relink under `interrupts::critical_section` only, without any state/ `parent_` recheck after the lock is dropped (the check is outside the section), so a concurrent `internal_exit_()` can interleave.

Impact (confirmed behaviour change): the previous stub returned `result::ok`; code that detaches a top-level thread now fails with `EINVAL`. On SMP a detach racing a termination can leave the thread on `top_threads_list_` after destruction (**suspected** dangling node).

Suggestion: treat an already-`top_threads_list_` thread as success, move the whole test/relink under one critical section, and re-check `state_ == terminated/destroyed`.

[MEDIUM] `waiting_threads_list::link` does not initialise a first-use list, unlike its siblings — `src/rtos/internal/os-lists.cpp:193`

```
void waiting_threads_list::link (waiting_thread_node& node)
{
    thread::priority_t prio = node.thread_>priority ();
    waiting_thread_node* after = static_cast<waiting_thread_node*> (
        const_cast<utils::static_double_list_links*> (tail ()));
    if (empty ()) { ... }
```

`ready_threads_list::link` (line 47) and `terminated_threads_list::link` (line 535) both begin with `if (head_.prev () == nullptr) clear ();`. `waiting_threads_list` inherits `utils::double_list`, whose constructor clears the list, so constructed objects are fine — but a BSS-zero list (the C API places some objects in user memory) would give `tail() == nullptr` and `insert_after()` would dereference null (`lists.cpp:136`).

Impact (confirmed inconsistency; latent null-deref): the invariant "`insert_after` is always given linked neighbours" is only maintained by construction, not enforced where its siblings enforce it.

Suggestion: add the same first-use `clear()` guard, or document and assert `!uninitialized()`.

[MEDIUM] `double_list_iterator` post-increment/decrement use member functions as data — `include/cmsis-plus/utils/lists.h:920, 928, 937`

```
node_ = static_cast<iterator_pointer> (node_>next);    // operator++(int)
...
node_ = static_cast<iterator_pointer> (node_>prev);    // operator--()
...
node_ = static_cast<iterator_pointer> (node_>prev);    // operator--(int)
```

`static_double_list_links` declares `next_ / prev_` data members and `next() / prev()` functions; `node_>next` names the member function, and casting a pointer-to-member-function to `N*` is ill-formed. The prefix forms (lines 911, 928) correctly call `next() / prev()`. These templates are only instantiated for `waiting_threads_list`, whose `begin()` is not used by the kernel today, so the defect is latent.

Impact (confirmed latent compile break / wrong code if instantiated): anyone using post-increment/decrement on a `waiting_threads_list` iterator fails to compile (or is silently mis-handled if it ever compiled).

Suggestion: `node_ = static_cast<iterator_pointer> (node_>next ());` etc.

[MEDIUM] Test-runner log greps can declare false verdicts and leak the tee process — `test_smpl/run-hw.sh:239, 242; test_smpl/run-host.sh:58; test_smpl/run-qemu.sh:89`

```
if grep -qE 'Invalid ACK|Polling failed|Examination failed' "$log"; then rc=6; break; fi
if grep -qE '^Error: ' "$log"; then rc=5; break; fi
...
"$OPENOCD" "${args[@]}" > >(tee "$log") 2>&1 &
local ocd=$!
...
kill "$ocd"; wait "$ocd"
```

`^Error:` also matches benign OpenOCD errors (a missing optional target, a flash probe warning) that do not abort the run, turning a passing test into `OPENOCD ERROR`. The `kill / wait` pair reaps OpenOCD but not the `tee` process substitution, so the log can still be written after the verdict is read. In `run-host.sh:58` and `run-qemu.sh:89`, `exit 2` on the first non-executable `*-host / *-qemu.bin` entry aborts the whole suite instead of skipping it (a stray non-executable file is enough).

Impact (confirmed robustness): flaky CI verdicts and occasional truncated log reads; a single stray file aborts a multi-test run.

Suggestion: match only fatal OpenOCD diagnostics (or gate on the process having exited first), wait on the `tee` PID as well, and `continue / skip` rather than `exit` per-entry.

[MEDIUM] Memory allocators are only safe when externally locked — `src/memory/first-fit-top.cpp:120`; `src/memory/block-pool.cpp:48`; `src/memory/lifo.cpp:70`

`first_fit_top::do_allocate/do_deallocate`, `block_pool::do_allocate/do_deallocate` and `lifo::do_allocate` mutate free lists/statistics with no internal critical section. They are safe only because the paths that are actually used (`operator new` in `src/libcpp/new.cpp:145`, and `allocator_stateless_polymorphic_synchronized` in `os-memory.h:1575`) wrap calls in a scheduler/kernel lock.

Impact (suspected misuse hazard): any direct `resource.allocate()/deallocate()` from two cores (the `pmr` `memory_resource` interface is public) corrupts the free list. The classes' own docs do not warn about this for `first_fit_top`.

Suggestion: either document "not thread safe; lock externally" on `first_fit_top` / `lifo` / `block_pool`, or add an internal lock.

[LOW] `first_fit_top` / `memory_pool` use relational comparison on unrelated pointers — `src/memory/first-fit-top.cpp:259`; `src/rtos/os-mempool.cpp:826`

```
if ((addr < arena_addr_) || (addr > (static_cast<char*> (arena_addr_) + total_bytes_)))
```

Relational comparison of `void*` / `char*` values that are not part of the same array is unspecified/UB in C++. Also the upper bound uses `>` where `>=` is the correct exclusive limit (the chunk metadata at `arena_end - chunk_offset` is not a valid payload).

Impact (confirmed portability nit; benign on flat embedded maps).

Suggestion: compare `reinterpret_cast<uintptr_t>` values and use `>=` for the high end.

[LOW] Semihosting `lseek` narrows a 64-bit offset to `int` — `src/semihosting/c-syscalls-semihosting.cpp:188, 210, 246`

```
static int __semihosting_lseek (int fd, int ptr, int dir) ...
ptr = pfd->pos + ptr;
...
pfd->pos = ptr;
```

`__posix_lseek` passes an `off_t`, and `pfd->pos` is `int`. Files ≥ 2 GiB cannot be addressed, and `pos + ptr` can overflow signed `int` (UB).

Impact (confirmed on large files; Fine for small embedded images).

Suggestion: keep the semihosting `fh` / position in `off_t` (or `long long`) and range-check before the host `SYS SEEK` call.

[LOW] `__posix_system` has a dead `exit_code` and a dubious shift loop — `src/semihosting/c-syscalls-semihosting.cpp:606`

```
int e = __semihosting_checkerror (call_host (SEMIHOSTING_SYS_SYSTEM, block));
if ((e >= 0) && (e < 256))
{
    int exit_code;
    for (exit_code = e; (e != 0) && (WEXITSTATUS (e) != exit_code); e <<= 1) { continue; }
}
return e;
```

The loop computes nothing observable (`exit_code` is unused, `e` may be shifted out), so the intended exit-status normalisation is a no-op.

Impact (confirmed): `system()` returns the raw host value despite the comment claiming conversion.

Suggestion: implement the newlib exit-status encoding or delete the dead loop and use `WEXITSTATUS`.

[LOW] `__posix_getcwd` ignores `size` semantics — `src/semihosting/c-syscalls-semihosting.cpp:669`

```
char* __posix_getcwd (char* buf, size_t size)
{
    strncpy (buf, "/tmp", size);
    return buf;
}
```

If `size < 5` the result is not NUL-terminated; `strncpy(..., 0)` writes nothing. POSIX requires `NULL` + `ERANGE` when the buffer is too small. `buf == nullptr` also faults.

Impact (confirmed): caller buffer overrun / non-terminated string for small buffers.

Suggestion: `if (buf == nullptr || size < 5) { errno = ERANGE; return nullptr; }` then copy.

[LOW] `file_descriptors_manager::valid()` returns true for in-range but unallocated fds — `src/posix-io/file-descriptors-manager.cpp:106`

```
bool file_descriptors_manager::valid (int fildes)
{
    if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__)) return false;
    return true;
}
```

It only range-checks, unlike `io(fildes)` which also checks the slot and `descriptors_array__ != nullptr`. A caller using `valid()` as "is this an open descriptor" gets the wrong answer.

Impact (confirmed semantic bug): stale/never-opened fds pass a validity test.

Suggestion: have `valid()` test `descriptors_array__ != nullptr && descriptors_array__[fildes] != nullptr`.

[LOW] `osMessageGet` leaves `osEvent` fields uninitialised — `src/rtos/os-c-wrapper.cpp:4721`

```
osEvent event;
...
event.status = ...;    // value.v / value.p / def set only on some paths
return event;
```

`osEvent.def` is never assigned and `value` is only written on the message path; the non-message paths return a partially-uninitialised aggregate to the caller.

Impact (confirmed): callers reading `event.def` / `event.value.p` on a timeout or error get indeterminate data.

Suggestion: value-initialise `osEvent event{}`; and set `event.def` / `value` as the CMSIS API specifies.

[LOW] Fallback to a null idle thread in the SMP picker — `src/rtos/os-core.cpp:623`

```
else
{
    scheduler::current_thread[cpu] = scheduler::os_idle_thread_core[cpu];
    if (scheduler::os_idle_thread_core[cpu] != nullptr)
        scheduler::os_idle_thread_core[cpu]->state_ = thread::state::running;
}
instrumentation::thread::active (scheduler::current_thread[cpu]);
```

During the boot window a secondary core can have no registered idle thread; the picker then installs `nullptr` as the current thread and passes it to instrumentation. The ports abort on a null `new_thread` after this call (e.g. `aarch64/src/rtos/os-core.cpp`), and `instrumentation::thread::active(nullptr)` may dereference.

Impact (suspected, boot-window only): null dereference if instrumentation is enabled before the secondary idle thread is registered.

Suggestion: guard `instrumentation::thread::active(...)` with a null check, or keep the per-core fake/boot thread current until the idle thread is registered.

[LOW] `os-c-wrapper` comment contradicts the `try_receive` result contract — `src/rtos/os-c-wrapper.cpp:4746`

```
.resume ... try_receive (... , nullptr);  
// result::event_message when message;  
// result::ok when no message
```

`message_queue::try_receive` actually returns `result::ok` when a message was received and `EWOLDBLOCK` when the queue is empty (`src/rtos/os-mqueue.cpp:1513-1518`). The code below it maps `result::ok` → `osEventMessage` and `EWOLDBLOCK` → `osOK` , which is correct; only the comment is inverted. Documentation-only, but the same comments in `osMessagePut` / `osMessageGet` will mislead the next reader.

Impact (confirmed doc bug; no runtime effect).

Suggestion: correct the comment to match `os-mqueue.cpp` .

[NIT] `run-host.sh` / `run-qemu.sh` collect `results[]` but never use it — `test_smpl/run-host.sh:53`, `test_smpl/run-qemu.sh:84`

`declare -a results=()` is appended to for every verdict but never printed or returned. Harmless, but dead state that suggests a lost summary feature (a JUnit/machine report).

Suggestion: print the array in the summary, or drop it.

Good practices

- RAIL critical sections (`interrupts::critical_section` , `scheduler::critical_section` , `uncritical_section`) are used consistently, and the recursive per-core kernel lock in the ports makes nesting safe; the kernel's own `internal_switch_threads` / `internal_link_node` document their locking preconditions.
- The SMP life-cycle paths carry explicit, well-commented fixes: the `join()` reclaimer waits until no CPU has the target current (`os-thread.cpp:1099-1121`), `kill()` has the analogous gate with `port_smp_ipi()` (`os-thread.cpp:1442-1498`), and the idle reaper applies the same "off every CPU and context saved" rule (`os-idle.cpp:83-100`).
- The deferred publish/claim protocol is documented in the kernel comment at `os-core.cpp:589-592` and implemented with release stores in the ports; the ordering rationale for releasing the kernel lock word last is spelled out and was clearly learned from real deadlocks.
- The board contract file (`test_smpl/src/board-contract.cpp`) turns missing per-board facts into compile errors — an excellent pattern that prevented several silent defaults.
- Test runners give explicit per-test wall-clock budgets and classify PASS/SKIP/FAIL/TIMEOUT/NO-RESULT, and `run-hw.sh` refuses to run a suite because a power cycle is required between tests (`run-hw.sh:280`).

- `test_smpl/include/hw_result.hpp` centralises the hardware PASS/FAIL handshake so QEMU and silicon runs share one verdict format.

µOS++ Ille AArch32 / AArch64 port review

Scope

Read-only review of the two architecture ports and their board/test trees:

- `micro-os-plus-iii-aarch32.git`: `include/`, `src/` (`context_switch.cpp`, `exception_handler.cpp`, `handlers.cpp`, `rtos/os-core.cpp`, `semihosting-exit.cpp`, `smp_secondary.cpp`) and `test/boards/{rpi-zero-2w, luckfox-lyra}/**` (`startup.S`, `smp.cpp`, `mmu.cpp`, `timer_arm.cpp`, `uart.hpp`, `gic.hpp`, `rtos/port_isr.cpp`, `port_sys.cpp`, `USB/DWC2`).
- `micro-os-plus-iii-aarch64.git`: same top-level layout plus `test/boards/rpi-zero-2w/**`.

Supporting facts were cross-checked against the kernel (`micro-os-plus-iii-smp.git`: `src/rtos/os-core.cpp`, `os-sched.h`, `port/smp-common/.../os-decls.h`) and the BCM2837 device headers. No files were modified; this report is the only artifact written.

Summary

The two ports share a genuinely careful SMP design: a single recursive LDREX/STREX (AArch32) / LDAXR/STLXR (AArch64) kernel lock, a deferred "publish" so a switched-out `stack_ptr` is only visible after SP has left the outgoing stack, a claim-by-nulling protocol in `internal_switch_threads`, a tripwire that validates the inbound frame, and a fault console that deliberately avoids the semihosting trap. The 64-bit and 32-bit variants are structurally parallel and the divergences are mostly intentional ISA splits.

The most consequential defect found is a **timing inconsistency on the Raspberry Pi AArch32 port**: the kernel's high-resolution clock reads the raw `CNTFRQ` (`timer_arm::get_freq()`) even though the board's own code documents that this value is wrong and calibrates the real rate into `frequency()`; the two must be read from the same place. Other real issues are the AArch64 MMU leaving TTBR1 walks enabled with `T1SZ=0`, the USB gadget stack compiling only at `-O0`, a misleading `OS_HAS_INTERRUPTS_STACK` declaration, silent secondary-release timeouts, and several smaller attribute/type divergences. No remotely-triggerable memory-corruption or data-race defect was confirmed in the scheduler lock or the publish protocol itself.

Findings

[High] High-res clock uses the known-wrong CNTFRQ while the tick uses the calibrated rate — `include/cmsis-plus/rtos/port/os-inlines.h:309` (and `:315`)

```
clock_highres::input_clock_frequency_hz (void)
{
    return timer_arm::get_freq();
}
...
clock_highres::cycles_per_tick (void)
{
    return timer_arm::get_freq() / 1000;
}
```

On the Pi Zero 2W the board explicitly refuses to trust `CNTFRQ` and measures the counter instead (`test/boards/rpi-zero-2w/src/timer_arm.cpp:29-59`, `frequency() / period_cycles()` use the calibrated `g_freq`), yet the entire `clock_highres` path uses `timer_arm::get_freq()`, which is the raw `mrc p15,0, ..., c14, c0, 0` read (`include/timer_arm.hpp:41-45`). The file's own comment says `CNTFRQ` "reports 19.2 MHz while the physical generic-timer counter actually increments at ~1 MHz". Consequently `std::chrono / hrclock` durations derived from `input_clock_frequency_hz()` are wrong by that ratio. Confirmed defect (latent until an app uses the high-res clock). Fix: have `clock_highres::input_clock_frequency_hz() / cycles_per_tick()` call `timer_arm::frequency()` (the calibrated value), or make the board override `get_freq()`.

[Medium] AArch64 MMU leaves TTBR1 enabled with T1SZ=0 — `test/boards/rpi-zero-2w/src/mmu.cpp:139`

```
std::uint64_t tcr =
    (30ULL << 0) | // T0SZ
    ...
    (2ULL << 32); // IPS = 40-bit
```

`TCR_EL1` is written once with `T1SZ = 0` and `EPD1 = 0`, so TTBR1 table walks remain enabled over the whole 64-bit space, overlapping the TTBR0 region; `TTBR1_EL1` itself is never programmed. All current code uses low VAs so TTBR1 is not selected, but any negative/interior 64-bit pointer (a common consequence of an ABI bug) turns into an unpredictable walk through an uninitialised TTBR1 instead of a clean translation fault. Suspicion/robustness (not observed). Fix: set `EPD1 = 1` (and `T1SZ`), or program TTBR1.

[Medium] DWC2 gadget stack only enumerates at `-O0` — `test/luckfox-lyra/`

`tests.cmake:162`

```
# This is not a preference and not a workaround we could drop: the source under
# usb/ is BYTE-IDENTICAL ... but only at -O0.
...
PROPERTIES COMPILE_OPTIONS "-O0"
```

The comment itself records that at `-O2` the EP0/descriptor path is "rewritten into something the hardware will not enumerate with". That is the signature of optimisation-sensitive code (missing `volatile`, missing barrier, or relying on statement ordering/timing) rather than a toolchain preference; the workaround hides a latent compiler-dependence. Confirmed as a build-fragility defect (the issue is documented, the root cause is not fixed). Fix: identify the `volatile` / `asm` ordering the `-O2` build violates; do not rely on `-O0`.

[Medium] AArch32 SYS_EXIT passes a bare reason word where the Angel/OpenOCD ABI is a `{reason, status}` block — `include/semihosting.hpp:108`

```
register const int* r1 asm ("r1") = &code;
asm volatile ( UOS_SEMIHOST_TRAP ... );
```

The hardware (non-`QEMU_BUILD`) AArch32 path hands the debugger a pointer to a single `int` (the reason). The AArch64 sibling correctly passes a two-field block (`include/semihosting.hpp:67-73` in the aarch64 repo: `volatile uint64_t block[2] = { reason, status };`). Implementations that follow the modern `{reason, subcode}` parameter block read the second word from adjacent stack memory, so the reported exit status is garbage; only the printed `RESULT:` line saves the run. Suspicion (ABI is debugger/version-dependent), but the asymmetry with the AArch64 port is confirmed. Fix: pass a two-word block, matching AArch64.

[Medium] Secondary-core release timeout is silently ignored — `test/boards/rpi-zero-2w/src/smp.cpp:63` (AArch32) and `test/boards/rpi-zero-2w/src/smp.cpp:68` (AArch64)

```
for (unsigned spin = 0; spin < 200000000u; ++spin) {
    __asm__ volatile("dmb" ::: "memory");
    if (_port_core_alive[core])
        break;
    ...
}
```

If a secondary never ticks (bad mail-box write, missing timer route, cache coherency handoff failure), `release_one()` gives up and `start_secondary_cores()` releases the next core anyway, with no error surfaced. The failure then appears later as a hang or a wrong `g_core_stage` in an unrelated test. The AArch64 version at least prints a `DEBUG_BOOT`-only message (`test/boards/rpi-zero-2w/src/`

`smp.cpp:75-78`). Confirmed. Fix: make the timeout a hard error reported through the board console (and the semihosting result).

[Medium] AArch64 port ignores `OS_SYSTICK_DIV` that AArch32 supports — `test/boards/rpi-zero-2w/src/rtos/port_isr.cpp:65`

```
uint32_t freq = timer_arm::get_freq();
timer_arm::set_tval(freq / 1000);
```

The AArch32 port re-arms with `timer_arm::period_cycles()` and phases the global tick by `OS_SYSTICK_DIV` (`aarch32 .../port_isr.cpp:88-101`, `include/timer_arm.hpp:25-27`). The AArch64 port hard-codes `/1000` and has no `OS_SYSTICK_DIV` handling, so a test built with `-DOS_SYSTICK_DIV=2` gets 0.5 ms *logical* ticks (twice the intended sysclock rate) instead of 0.5 ms preemption with a 1 ms sysclock. Confirmed divergence.

[Medium] `OS_HAS_INTERRUPTS_STACK` is declared but the full ISR frame lives on the task stack — `include/cmsis-plus/rtos/port/os-c-decls.h:76` (AArch32) and `:67` (AArch64)

Both ports define `OS_HAS_INTERRUPTS_STACK`, which makes the kernel print and expose a separate interrupt stack, but the AArch32 IRQ handler switches to SVC mode and builds the entire 82-word context frame on the *task's* SVC stack (`src/handlers.cpp:168-192`); only `r0-r3` briefly touch the IRQ stack. The AArch64 linker comment admits the region is "only what μ OS++ records for its bookkeeping" (`aarch64/linker.ld:64-66`). A deep ISR therefore consumes task stack, and the capability advertised to the kernel is not real. Confirmed. Fix: drop the macro, or actually use the interrupt stack for the frame.

[Medium] Pi timer calibration runs on every core with IRQs masked — `test/boards/rpi-zero-2w/src/timer_arm.cpp:50`

```
void init() noexcept {
    g_freq = calibrate_hw_freq();    // busy-waits ~50 ms
    ...
}
```

`port_sys_init()` calls `timer_arm::init()` on every core (`test/boards/rpi-zero-2w/src/port_sys.cpp:33`), and calibration busy-waits 50 ms against the BCM system timer with interrupts masked. That is ~150 ms of extra boot latency and turns a hardware-fragile measurement into four independent measurements of a *shared* counter. Confirmed (perf/robustness). Fix: calibrate once on core 0 and publish the result.

[Medium] AArch32 board-fact contract not enforced in the AArch64 port —

[aarch64/include/cmsis-plus/rtos/port/os-c-decls.h:52](#)

```
#ifndef OS_NCPU
#define OS_NCPU      4
#endif
...
#define OS_SMP_IPI_SGI 0
```

The AArch32 port deliberately `#error`s unless `OS_NCPU` / `OS_SMP_IPI_SGI` come from `board.cmake` (`aarch32 .../os-c-decls.h:58-63`), precisely so a board cannot silently get the wrong core count. The AArch64 variant re-introduces silent defaults. Confirmed divergence; the stated contract/logic is weaker on the 64-bit port. Fix: mirror the `#error` contract.

[Low] Deferred publish has no release barrier before the pointer store — [test/](#)

[boards/rpi-zero-2w/src/startup.S:117-131](#) (AArch64) and [aarch32/src/handlers.cpp:199-216](#)

`SMP_PUBLISH` stores `old->stack_ptr` and only then issues `dmb ish`. Frame visibility to a claimant core is actually guaranteed by the kernel lock's release-acquire pairing (the frame is written before `_smp_klock` is released; the claimant acquires the lock before reading the frame), so the current code is correct. But the publish store itself is not ordered as a release, so any future change to when/how the lock is released would silently expose a partially invisible frame. Suspicion/robustness. Fix: use `stlr` (or a `dmb` before the store) and document the dependency.

[Low] AArch32 TTBR0 inner attribute is WB, not WBWA as the comment claims —

[test/boards/rpi-zero-2w/src/mmu.cpp:96](#)

```
uint32_t ttbr = v | (1u << 6) | (1u << 3) | (1u << 1) | (1u << 0);
```

IRGN is encoded as `{bit6, bit0}`; setting both bits selects write-back, not write-allocate, while the comment and the DRAM section mapping use WBWA (`kNormalwbwa`). Both are cacheable+shareable, so coherency and the exclusive monitor still work; this is an attribute inconsistency, not a corruption. Confirmed. Fix: set only bit 6 for IRGN=WBWA.

[Low] AArch64 spin-table release base may be off by one — [test/boards/rpi-zero-2w/](#)

[src/smp.cpp:34](#)

```
return reinterpret_cast<volatile std::uint64_t*>(
    static_cast<std::uintptr_t>(0xD8u + 8u * core));
```

The upstream BCM2837 device tree places CPU1's release word at `0xD8` (formula `0xD8 + 8*(core-1)`), whereas this code writes CPU1 to `0xE0`. The AArch32 shim uses the same `0xD8 +`

`8*core` formula and is reported to work, so either the firmware/QEMU stub here differs from the DT layout or the hardware path never uses these words (tests resume all cores via OpenOCD, `hw.sh` `UOS_HW_RESUME=pc`). Cannot confirm without hardware; flagged as a verification item, not a proven bug.

[Low] AArch32 luckfox linker region overruns the mapped DRAM top — `test/`

`boards/luckfox-lyra/linker.ld:29` VS `src/mmu.cpp:39`

```
RAM (rwx) : ORIGIN = 0x00200000, LENGTH = 0x08000000 /* ends 0x08200000 */
```

```
constexpr uint32_t kDramTop = kDramSizeMb << 20; // 0x08000000
```

The linker believes RAM extends to `0x08200000` while the MMU only maps DRAM below `0x08000000`; anything the linker might place in that 2 MiB tail would translation-fault. Nothing currently lands there, so it is latent. Confirmed inconsistency.

[Low] `udelay` / `get_timer` hard-code 24 MHz, decoupled from `timer_arm` —

`test/boards/luckfox-lyra/usb/src/usb_env_stateos.cpp:36`

```
constexpr std::uint32_t kCntHz = 24000000u; // CNTFRQ set by startup.S
```

`timer_arm::init()` resolves the rate (falling back to 24 MHz only when `CNTFRQ==0`, `test/boards/luckfox-lyra/src/timer_arm.cpp:24-31`), but the USB environment shim hard-codes the same number. They agree today; if `CNTFRQ` ever comes back non-zero-but-different (e.g. a different loader) USB timing silently diverges from the RTOS's. Also `mdelay(ms)` computes `ms * 1000u` in 32-bit `unsigned long`, which overflows for large delays (`usb_env_stateos.cpp:83`). Confirmed.

[Low] AArch32 secondary start has dead code after `reschedule()` — `src/rtos/os-core.cpp:424`

```
os::rtos::port::scheduler::reschedule ();  
__asm__ volatile("cpsie i" ::: "memory");  
for (;;) { __asm__ volatile("wfi"); }
```

`reschedule()` performs the first `port_ctx_switchHandler()` and never returns, so the interrupt-enable and `wfi` loop are unreachable. Harmless, but it obscures that the secondary depends on the first switch completing. Confirmed (dead code).

[Low] AArch32 `switch_stacks` re-implements `_smp_klock_raw_acquire` inline —

`src/rtos/os-core.cpp:213`

The LDREX/STREX acquisition in `switch_stacks` duplicates the exact instruction sequence of `_smp_klock_raw_acquire()` (`os-inlines.h:70-86`), as does the release logic vs `_smp_klock_raw_release()`. Two copies of a memory-ordering primitive is how the release order regressed before (the comment at `os-core.cpp:302-319` documents a prior SMP freeze from exactly this). Confirmed maintainability/divergence risk; fix: call the helper.

[Low] AArch64 tripwire hard-codes the SPSR offset — `src/rtos/os-core.cpp:249`

```
uint64_t spsr = (p != nullptr) ? *reinterpret_cast<uint64_t*> (p + 248) : 0;
```

`port_ctx.hpp:28` already exports `SPSR_IDX = 248/8` for exactly this purpose, but `os-core.cpp` uses the literal `248` (and the AArch32 tripwire correctly uses `ctx_cpsr_word`). A layout change would silently desynchronise the AArch64 check. Confirmed. Fix: use `SPSR_IDX * sizeof(uint64_t)`.

[Low] Stale/contradictory board comments — `aarch32/test/boards/rpi-zero-2w/src/`

`startup.S:9`, `include/uart.hpp:33`, `aarch64/include/cmsis-plus/os-app-config.h:7`

- `startup.S:9` says "QEMU raspi3b: ALL four cores are started at `_start` simultaneously", but `qemu-raspi3-shim/shim.S` releases cores 1-3 from QEMU's spin-table and `qemu.sh` loads via the shim.
- `uart.hpp:33` says "SEMIHOST is defined only by the hardware-run build", but `test/CMakeLists.txt:66` puts `SEMIHOST` in `_common_defines` for every variant; the same claim is repeated in `include/semihosting.hpp:5`.
- `aarch64/include/cmsis-plus/os-app-config.h:7` still describes "the RK3506 Cortex-A7 SMP port ... OS_NCPU=3" in the AArch64 repo.

All are documentation defects (Nits individually), but the first two can send a bring-up investigation down the wrong path.

[Low] AArch32 timer accessors omit `"memory"` clobbers — `include/`

`timer_arm.hpp:37-65`

`set_freq`/`get_freq`/`set_tval`/`get_ctl` etc. are `__asm__ volatile` MMIO operations with no `"memory"` clobber, so the compiler may move ordinary loads and stores across them. The engine's other register accessors in this tree do add `"memory"`. Confirmed (theoretical reordering); fix: add `"memory"`.

AArch32 vs AArch64 divergences

Beyond the ISA-mandated ones (banked modes/VFP vs PSTATE/Q-registers, CP15 vs system registers, GIC vs the BCM local controller), the following are semantic and worth reconciling:

Topic	AArch32	AArch64
<code>OS_SYSTICK_DIV</code>	supported (<code>period_cycles()</code> , phase counter)	ignored, <code>freq/1000</code> hard-coded
Board-fact contract	<code>#error</code> if <code>OS_NCPU</code> / <code>OS_SMP_IPI_SGI</code> unset	silent defaults (<code>OS_NCPU=4</code> , SGI 0)
Kernel lock asm	<code>LDREX/STREX</code> + <code>dmb</code>	<code>LDAXR/STLXR</code> + <code>dmb ish</code> (fine)
Release barrier	<code>dmb</code> / <code>dsb</code> (full system)	<code>dmb ish</code> / <code>dsb ish</code>
<code>switch_stacks</code> acquire	inline duplicate of helper	uses <code>ldaxr/stlxr</code> inline too
SYS_EXIT parameter	bare <code>int*</code> on hardware	<code>{reason,status}[2]</code> block
Tripwire SPSR offset	<code>ctx_cpsr_word</code> constant	literal <code>248</code>
Fault reporting	rich abort/undef dumps + test-mode resume	minimal ESR/ELR/FAR dump, no resume
<code>_Exit</code> location	separate <code>semihosting-exit.cpp</code>	inside <code>handlers.cpp</code>
<code>input_clock_frequency_hz</code>	uses wrong raw <code>get_freq()</code> (Pi)	raw <code>get_freq()</code> is honest
<code>port_cpu_id</code> mask	<code>&3</code> (port_sys) / <code>&0xFF</code> (inlines)	<code>&0xFF</code>
Interrupt frame stack	task SVC stack; IRQ stack only for r0-r3	task SP_EL1 only

Good practices

- **Deferred publish + claim-by-nulling.** The `_smp_pub_addr/_smp_pub_val` staging, the `stack_ptr = nullptr` claim, and the `th == old_thread || (state != running && stack_ptr != nullptr)` skip in `internal_switch_threads` (`smp/src/rtos/os-core.cpp:600-606`) correctly prevent a second core from running a thread whose stack is still in use.
- **Release ordering documented and implemented.** `switch_stacks` clears `depth` / `owner` before the lock word and explains the prior total-freeze bug (`aarch32/src/rtos/os-core.cpp:302-322` , aarch64 `:278-299`).
- **Tripwires.** Frame-bounds + CPSR/`SPSR` mode validation before every switch (`os-core.cpp:244-287` , aarch64 `:238-268`) turns a protocol violation into an on-the-spot report.
- **Fault console isolation.** `FaultConsole` prints only through `puts_uart` / `putc_uart` , with an excellent rationale for why a fault handler must not touch the semihosting trap (`aarch32/src/exception_handler.cpp:15-37`).
- **Cache maintenance for a caches-off receiver.** `DCCMVAC` / `dc cvac` + `dsb` before `sev` /mailbox release (`aarch32/.../smp.cpp:49-60` , `aarch64/.../smp.cpp:53-59`), with the coherency reasoning in comments.
- **Semihosting trap is treated as a property of the debugger**, selected per board, and the HLT-on-ARMv7 hazard is called out and avoided for the A7 (`include/semihosting.hpp:10-34`).
- **Board-fact centralisation** in `board.cmake` (`OS_NCPU` , `PORT_RAM_*` , `PORT_GREETING`) with `#error` enforcement on the AArch32 side.

Code review — μOS++ Ille Cortex-M and POSIX architecture ports

Scope

Read-only review of two working copies under `/home/dan/Work`:

- `micro-os-plus-iii-cortexm.git`
- the port proper: `include/`, `include-m33/`, `include-rp2350/`, `src/rtos/os-core.cpp`, `src/rtos/os-core-m33.cpp`, `src/rtos/os-core-rp2350.cpp`, `src/semihosting-exit.cpp`, `src/libc/getentropy.c`, `CMakeLists.txt`;
- the board/test trees: `test/boards/pico2/*`, `test/boards/pico2-pizero/*`, `test/boards/pico2-rp2350b-psram/*`, `test/boards/shared/*`, the STM32 boards' glue and the `hw.sh`/`qemu.sh` runners.
- `micro-os-plus-iii-posix-arch.git`
- `include/*`, `include/cmsis-plus/rtos/port/*`, `src/*` (`host_cpu.cpp`, `rtos/os-core.cpp`, `exception_handler.cpp`, `free-store.cpp`, `board-contract.cpp`, `diag/trace-posix.cpp`);
- `test/boards/native/*`, `test/run.sh`.

Sibling repos (`micro-os-plus-iii-smp.git`, `micro-os-plus-iii-devices.git`) were consulted only to establish the kernel-side contracts the ports must honour (`internal_switch_threads()`'s `stack_ptr != nullptr` pick rule, `scheduler::lock()/unlock()` semantics, `exit()` → `_Exit()`). No file was modified except this report.

Summary

The ports are unusually well documented and honest about their design; the SMP model (deferred publish, per-core tick, recursive kernel lock) is coherent and the comments correctly explain the hard parts. The problems below are concentrated in the **divergence between the two Cortex-M SMP cores** (`os-core-m33.cpp` vs `os-core-rp2350.cpp`): the RP2350 copy lost the M33's PRIMASK save/restore, and both copies lost the upstream Cortex-M port's local interrupt masking in `switch_stacks()`. The high-resolution-clock overflow test is wrong (dead code) in all three headers. The POSIX port's most serious portability defect is the macOS tick, which is process- not thread-directed.

No finding here is demonstrated by a failing test in this read-only pass; classifications below separate defects I can prove from the code from suspicions that need a run to trigger.

Findings

[Critical] SMP "no ready thread" path spins forever while holding the kernel spinlock — `src/rtos/os-core-m33.cpp:471`, `src/rtos/os-core-rp2350.cpp:448`

```
if (new_thread == nullptr)
{
    __asm__ volatile ("cpsid if" ::: "memory");
    for (;;)
        __asm__ volatile ("wfi");
}
```

`switch_stacks()` acquired `_smp_klock` immediately above (m33 lines 455–460, rp2350 lines 426–431) and never releases it on this path. If it is ever taken, the other core spins forever in the first critical section it enters — a silent, total SMP freeze rather than a diagnosable abort. It should clear `owner` / `depth` and release the lock (or abort) before spinning. The same defect exists verbatim in both cores.

[High] RP2350 scheduler unlock discards the caller's PRIMASK — `src/rtos/os-core-rp2350.cpp:352`

```
lock_state[cpu] = state;
port_put_lock (0);
```

The M33 core stores the PRIMASK captured when the lock was taken and restores it (`lock_primask[cpu]`, `src/rtos/os-core-m33.cpp:395,408`). The RP2350 core has no such array and passes a hard-coded `0`, so `port_put_lock()` ends with `__set_PRIMASK(0)` and unconditionally re-enables interrupts. The M33 header warns about exactly this (`include-m33/.../os-inlines.h:164-169`: "passing a hardcoded 0 would wrongly re-enable IRQs when the lock is taken with interrupts already disabled"). Any caller that enters the scheduler lock with IRQs masked and relies on them staying masked is silently broken. This is a direct consequence of the copy divergence noted below.

[High] SMP `switch_stacks()` does not mask local interrupts — `src/rtos/os-core-m33.cpp:449`, `src/rtos/os-core-rp2350.cpp:419`

The upstream Cortex-M port disables BASEPRI/PRIMASK for the duration of the picker (`src/rtos/os-core.cpp:842-869`). Both SMP cores instead run the whole context switch with only the cross-core spinlock and with interrupts enabled:

```

stack::element_t*
switch_stacks (stack::element_t* sp)
{
    #if defined(OS_USE_SMP_SCHEDULER)
        unsigned cpu = port_cpu_id ();
        if (_smp_klock.owner != cpu) { _smp_klock_raw_acquire (); ... }
    #endif
}

```

Because PendSV is the lowest-priority exception, every other IRQ can preempt it in the middle of `internal_switch_threads()`'s walk of `ready_threads_list_`. An ISR that calls a kernel API (semaphore post, timer, message queue) enters `critical_section`, and since `port_set_lock()` sees `owner == cpu` it does *not* spin — it recurses and mutates the same ready list the suspended picker is walking. That is exactly the corruption the upstream mask prevents. IRQs should be masked around the switch (or the ISR path must be forbidden, which it is not).

[Medium] High-resolution clock overflow test reads the wrong register — include/cmsis-plus/rtos/port/os-inlines.h:463, include-m33/.../os-inlines.h:397, include-rp2350/.../os-inlines.h:511

```

if (SysTick->CTRL & SCB_ICSR_PENDSTSET_Msk)

```

`SCB_ICSR_PENDSTSET_Msk` is bit 26 of `SCB->ICSR`; bit 26 of `SysTick->CTRL` is reserved and reads 0. The condition is therefore never true and the whole overflow-compensation block (`(load_value + 1 + (load_value - val))`) is dead code. The intended source is almost certainly `SCB->ICSR & SCB_ICSR_PENDSTSET_Msk` (the comment says "If the exception is pending"). Effect: `cycles_since_tick()` can under-report by up to one full period when the tick is pending but not yet serviced, degrading the high-resolution clock. Present identically in all three port headers.

[Medium] The two Cortex-M SMP cores are near-duplicates that have already diverged — src/rtos/os-core-m33.cpp, src/rtos/os-core-rp2350.cpp, include-m33/, include-rp2350/

`CMakeLists.txt:186-194` states the files are "91% the same file, and one day they should be one". They are not merely duplicated: they have diverged in ways that matter. The m33 core carries the `lock_primask[]` fix; the rp2350 core does not (see the High finding above). The m33 core launches core 1 from `start()` via `port_smp_launch_core1()`; the rp2350 core expects the test to call `multicore::launch_core1()` and exposes `port_core1_stack_top()`. The m33 core defines its switch instrumentation globals itself (`os-core-m33.cpp:81-87`); the rp2350 core declares them `extern` and expects the board (`os-core-rp2350.cpp:43-49`). Keeping two copies of a context-switch critical section is how the PRIMASK bug survived; the split should be closed or the shared body factored out.

[Medium] The RP2350 port headers depend on board BSP headers — include-rp2350/.../os-inlines.h:35, src/rtos/os-core-rp2350.cpp:39-40

```
#if defined(OS_USE_SMP_SCHEDULER)
#include <bsp/rp2350.hpp>
#endif
```

```
#include <bsp/multicore.hpp>
#include <bsp/rp2350.hpp>
```

The port's own `src/rtos/` and `include-rp2350/` cannot be compiled without the board's BSP include directory, inverting the stated layering ("cortexm gains SMP by a board arriving that supplies a lock and an IPI, not by this file changing", `CMakeLists.txt:29-34`). Any RP2350 board that names a different BSP header path will not build the port. The SIO/PSM/TICKS register definitions should be provided through a documented port-side interface rather than by direct inclusion of `bsp/`.

[Medium] POSIX macOS tick is process-directed, not per-CPU — src/host_cpu.cpp:220-224

```
#if defined(__linux__)
    sev.sigev_notify = SIGEV_THREAD_ID;
    sev._sigev_un._tid = static_cast<int> (::syscall (SYS_gettid));
#else
    sev.sigev_notify = SIGEV_SIGNAL;
#endif
```

The comment above this block explains precisely why `SIGEV_SIGNAL` is wrong ("delivers SIGALRM to an ARBITRARY thread of the process ... the preemption of CPU 2 could be charged to CPU 0"). On `__APPLE__` (which the file compiles for) the code still takes that branch, so the per-CPU tick model is broken on macOS: all timers target the process, delivery is arbitrary/coalesced, and the "only CPU 0 advances the clock" invariant no longer holds deterministically. Either implement a per-thread mechanism on macOS or gate SMP support on Linux.

[Medium] POSIX fault reporter calls non-async-signal-safe functions — src/exception_handler.cpp:125, :170

```
emit (::strsignal (static_cast<int> (type)));
...
int n = ::backtrace (frames, 24);
```

The file's own contract (lines 32-34) is "uses write(2) and nothing else: no printf, no malloc, no locks." `strsignal()` may take the locale lock / allocate, and `backtrace()` may take the dynamic-loader lock on first use. A deadlock here is unrecoverable — the process is already faulting and the handler is the only diagnostic. `backtrace()` is acknowledged as a calculated risk in the comment; `strsignal()` is not.

[Medium] POSIX `reschedule()` reads the CPU id before masking — `src/rtos/os-core.cpp:412`

```
void reschedule (void)
{
    const unsigned cpu = port_cpu_id ();
```

Every other place that reads the CPU id in this port masks first, and documents why (`src/rtos/os-core.cpp:181-189`, `src/host_cpu.cpp:186-189`, and the comment in `switch_stacks()` at lines 282-300). Here the tick is still enabled, so a signal delivered between `port_cpu_id()` and the later `_port_ctx_pending[cpu] = 1` can switch this context to another host thread; when `reschedule()` resumes, `cpu` names the core it left. It then either marks the wrong CPU pending or tests the wrong `_smp_klock.owner`. This is a suspicion in the sense that the deferral branches are only reached when the lock is held (where callers usually mask), but the code is inconsistent with the port's own rule and should mask before the read.

[Low/Medium] POSIX critical section saves only the tick signal, not the IPI — `src/cmsis-plus/rtos/port/os-inlines.h:197`

```
return ::sigismember (&old, clock::signal_number ()) != 0;
```

The mask is a two-signal set (tick + IPI). `critical_section::exit()` restores with `state ? SIG_BLOCK : SIG_UNBLOCK` over *both* signals (line 216), so a caller that had only the IPI blocked will have it unblocked. The prior state of `ipi_signal_number()` should be captured and restored separately.

[Low/Medium] POSIX per-CPU timers are never deleted — `src/host_cpu.cpp:227`

```
timer_t timer;
if (::timer_create (CLOCK_MONOTONIC, &sev, &timer) != 0) { fatal (...); }
```

`timer_delete()` is never called, so each CPU leaks a POSIX timer for the life of the process. In a long-lived process that starts/stops scheduler instances this accumulates. (Only one scheduler instance exists per process today, so the practical impact is small.)

[Low/Medium] RP2350 IPI push can block inside an interrupt handler — src/rtos/os-core-rp2350.cpp:614, test/boards/pico2/glue/multicore.cpp:60

```
// Non-blocking: if the FIFO is full the target already has a pending IPI.
void port_smp_ipi (unsigned cpu) { ... multicore::fifo_push (0x4D53u); }
```

```
void fifo_push (std::uint32_t v) { while (!fifo_write_ready ()) { } reg(...) = v; ... }
```

`fifo_write_ready()` is checked in `port_smp_ipi`, but `fifo_push()` re-checks and busy-waits if the FIFO became full in between (the FIFO is shared by both cores). `port_smp_ipi()` is reachable from ISR context; a full FIFO there spins with interrupts masked until the peer drains it. The claim "non-blocking" is not guaranteed by the implementation.

[Low] RP2350 core-1 launch handshake has no timeout — test/boards/pico2/glue/multicore.cpp:126

```
unsigned seq = 0;
do
{
    std::uint32_t cmd = cmd_seq[seq];
    ...
    seq = (cmd == response) ? seq + 1 : 0;
}
while (seq < 6);
```

The readiness wait above is guarded (2 000 000 iterations), but this echo handshake is not. If core 1 misbehaves (or a FIFO word is lost), the launch livelocks forever. A bounded retry with a diagnostic would fail fast.

[Low] RAM-exec boot stub silently accepts a failed verify — test/boards/pico2/src/boot.S:88

```
stub_retry_bad:
    subs r8, r8, #1
    bne stub_retry
stub_done:
    movs r0, #0
    msr primask, r0
    ldr r0, =_reset_handler
```

After the 32-attempt budget is exhausted the stub falls through into `_reset_handler` and runs a possibly-corrupt copy of the image. On a board whose whole reason for this stub is marginal flash, silently executing mismatched code is the worst outcome; it should signal (LED/loop/semihost) instead.

[Low] `_getentropy()` returns a predictable stream — `src/libc/getentropy.c:28`

```
for (size_t i = 0; i < length; ++i) { p[i] = (uint8_t)(0xA5u ^ (uint8_t)i); }
```

The comment calls it a "Deterministic placeholder", which is honest, but it is named `entropy` and satisfies callers that genuinely need unpredictability (newlib `getentropy`, `arc4random` fallbacks). A test that trusts it can never detect real RNG problems; this should at least be a compile-time opt-in, not the default.

[Low] Fault reporter can deadlock on the UART software lock — `test/boards/pico2/glue/rtos-glue.cpp:116`, `test/boards/pico2/glue/uart.cpp:48`

`fault_report()` writes through `uart::write()`, which takes SIO spinlock 1. If the fault occurred while the faulting core held that spinlock (e.g. inside `uart::write`), the spin in `lock_uart()` never terminates and the beacon loop is never reached. Fault paths generally need a lock-free/raw console write.

[Low] m33 switch-instrumentation globals are owned by the port while rp2350's are owned by the board — `src/rtos/os-core-m33.cpp:81-87`

```
volatile void* g_sw_old;
volatile void* g_sw_new;
...
const char* g_sw_hist[8];
```

`os-core-rp2350.cpp:43-49` declares the same names `extern` and expects the board (`glue/rtos-glue.cpp`) to define them. The m33 core instead defines them itself, non-static and without `extern "C"`. Same names, two different owners, differing linkage — a hazard if the m33 port is ever linked with a board that also defines them.

[Low] `hw_result.hpp` reimplements the semihosting trap — `test/boards/shared/hw_result.hpp:41`

```
register unsigned r0 __asm__ ("r0") = 0x04u; // SYS_WRITE0
register const char* r1 __asm__ ("r1") = s;
__asm__ volatile ("bkpt 0xAB" : : "r" (r0), "r" (r1) : "memory");
```

`src/semihosting-exit.cpp:16-18` argues at length that the kernel's `cmsis-plus/arm/semihosting.h` is the single place that spells the trap. The test helper nonetheless hardcodes `bkpt 0xAB`, so the port now has two sources of truth for the semihosting ABI. It should call the kernel helper.

[Low] POSIX runner delegates to an out-of-tree script — test/boards/native/run.sh:26

```
UOS_RUN_ONLY="${1:-}" \
exec "$SMP_DIR/test_smp1/run-host.sh" "$BUILD/test"
```

The actual per-test execution, timeouts and pass/fail parsing live in the SMP repo and cannot be reviewed here. `$BUILD` is passed through even when it does not exist, so the failure mode is the external script's. The dispatcher (`test/run.sh`) itself is fine.

[Low] POSIX kernel-lock spin has no backoff on architectures other than x86/ARM — include/cmsis-plus/rtos/port/os-inlines.h:104

```
while (__atomic_exchange_n (&_smp_klock.lock, 1u, __ATOMIC_ACQUIRE) != 0u)
{
    #if defined(__x86_64__) || defined(__i386__)
        __builtin_ia32_pause ();
    #elif defined(__aarch64__) || defined(__arm__)
        __asm__ volatile("yield" ::: "memory");
    #endif
}
```

On any other host the loop is a tight atomic exchange with no hint — wasteful and potentially unfair to the lock holder under oversubscription, which the README explicitly supports (`OS_NCPU > 16` warning in `src/board-contract.cpp:39`).

[Suspicion] POSIX `switch_stacks()` acquires the kernel lock unconditionally — src/rtos/os-core.cpp:308

```
_smp_klock_raw_acquire ();
_smp_klock.owner = cpu;
_smp_klock.depth = 1;
```

Unlike `critical_section::enter()`, this does not check `owner == cpu` first. It is safe only because both current callers (`reschedule()` and `irq_epilogue()`) return early when this CPU already owns the lock. A future caller reached from inside an `interrupts::critical_section` would self-deadlock on the spin. The recursive-entry guard belongs in `switch_stacks()` itself.

[Suspicion] POSIX high-res fallback shares `previous_timestamp` without synchronization — `src/rtos/os-core.cpp:561`

`static uint64_t previous_timestamp;` is plain storage read/written by `cycles_per_tick()` and `cycles_since_tick()`. Today only CPU 0 calls `cycles_per_tick()` (inside `os_systick_handler()`), and `has_hardware_counter()` returns true on this port so `hrclock::now()` uses `hardware_counter()` instead. If either assumption changes (a port where the fallback is live, or a caller on a non-zero CPU), this becomes an unsynchronized cross-CPU read-modify-write. It should be per-CPU or atomic.

[Suspicion] POSIX load-bias subtraction is meaningful only for this image — `src/exception_handler.cpp:131`

```
emit_hex (elr - g_load_bias);
```

`g_load_bias` is obtained from `dldaddr(&port_fatal_exception)`, i.e. the load base of the executable. Subtracting it from a return address that belongs to a shared object (libc, libstdc++) yields a meaningless "static" address. The `backtrace()` lines at 175 have the same problem. It is only cosmetic in a crash dump, but the label "static" overstates its accuracy.

Good practices

- The comments are exceptional: the SMP handover ordering, the deferred-publish rule, and the linker-order requirement for releasing owner/depth before the lock word (`src/rtos/os-core.cpp:360-373`) are documented with the failure mode they prevent, which made this review far faster.
- The Cortex-M generic core and the POSIX port both keep the `stack_ptr != nullptr` pick rule and the "mask before reading the CPU id" rule in one place, and cross-reference the AArch64/AArch32 equivalents.
- `src/semihosting-exit.cpp` correctly routes through the kernel's single trap definition, and `multicore.cpp:25-34` explicitly documents why `(void)reg(...)` is *not* enough to pop the SIO FIFO — a subtle, real C++ trap avoided.
- `psram.cpp`, `clocks.cpp` and `multicore.cpp` bound their hardware waits where it matters (PSRAM QMI, PLL lock, bootrom readiness), and the PSRAM init masks IRQs around the direct-mode window with an explicit rationale.
- The POSIX port uses `_exit()` in `hw_result.hpp` (with a `fflush`) deliberately to avoid running static destructors underneath live CPUs — the right call.
- The RP2350 reset handler releases all 32 SIO spinlocks (`test/boards/pico2/src/boot.S:127-135`) so a debugger reset cannot leave the next image spinning on a lock held by the previous one.
- `board.cmake` / `tests.cmake` discovery, the `OS_NCPU`-derived `OS_USE_SMP_SCHEDULER`, and the two-half board/ISA interface split are clean and make per-board differences auditable.

Read-only code review — µOS++ IIIe test harness / platforms / build system

Scope

Reviewed, read-only, under `/home/dan/Work/micro-os-plus-iii-smp.git`:

- `tests/CMakeLists.txt`, `tests/cmake/{global-definitions,common-options,tests-main}.cmake`
- `tests/platforms/*/` — every platform's `CMakeLists.txt`, `cmake/{definitions,platform-library,dependencies-folders}.cmake`, `src/platform-support.cpp` (5 copies), `include/`
- `tests/sources/*/` — the portable suites' CMake and sources (read for robustness)
- `tests/device-qemu-cortexm/CMakeLists.txt`
- `tests/package.json` (build configurations/actions)
- Harness docs: `tests/README.md`, `docs/tests/README.md`, `docs/tests/README-DEVELOPER.md`, `docs/tests/README-MAINTAINER.md`, and (for cross-checks) `docs/tests/HARNESS-TESTS-PARADIGM.md`, `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md`, `docs/tests/TESTS-CATALOG.md`
- Shell helpers invoked by the harness: `test_smpl/{run-qemu,run-hw,run-host}.sh`, plus the deprecated `tests/deprecated/scripts/*.sh`
- The port-owned builders/boards that the harness `add_subdirectory()`s were consulted only as evidence of the harness contract (they are separate repos).

No source files were modified. Existing build trees under `tests/build/` were inspected read-only; two existing QEMU images were re-run to test a suspicion (no tree change).

Summary

The harness is well-commented and the "port owns its tests, platform registers what the builder produced" seam is sound; the newer `cortexm-*`, `native`, `aarch32-*`, `aarch64-*` platforms follow it consistently. The most serious issues are (1) a real functional divergence in the Luckfox platform-support file that never releases the board's secondary cores, (2) a large amount of dead/misleading `platform/platform-support` CMake on the four Raspberry Pi platforms, hard-wired to the QEMU variant, and (3) build-hygiene gaps (toolchain guards only on aarch32/aarch64, an `ENABLE_HW_TESTS` flag that nothing reads, hard-coded sibling-repo paths). Documentation is partly acknowledged as historical, but `tests/README.md` and the two upstream READMEs are genuinely stale.

Two suspected defects were investigated and **disproved**; they are not listed: the `${rpath_options_list}` generator-expression is handled correctly by CMake (both `-wl, -rpath` and `-L` appear), and the recorded `2xcortex-m33-fp-switch-test` failure reproduces as PASS on re-run.

Findings

[High] Luckfox harness never releases the board's secondary cores — `tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp:94`

The Luckfox Lyra board is a 3-core SMP Cortex-A7 (`micro-os-plus-iii-aarch32.git/test/boards/luckfox-lyra/board.cmake:14` → `UOS_BOARD_NCPU 3`), and the platform sets `OS_USE_SMP_SCHEDULER=1` (`tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake:53`). Its harness trampoline, however, only installs the idle thread via `main()` and never brings up cores 1..2:

```
[[noreturn]] static void
harness_main_trampoline (void)
{
    int argc = 0;
    char** argv = nullptr;
    os_startup_initialize_args (&argc, &argv);
    int code = os_main (argc, argv);
    ...
}
```

The sibling AArch32 support files do it explicitly (`tests/platforms/aarch32-rpi-zero-2w/src/platform-support.cpp:106`):

```
smp_install_boot_threads ();
smp::start_secondary_cores ();
extern int test_wait_secondaries (int timeout_ms);
test_wait_secondaries (3000);
```

Confirmed in the configured build: the `mutex-stress-test-hwd` link line contains `.../luckfox-lyra/src/smp.cpp.obj` (which defines `start_secondary_cores`) and `platform-support.cpp.obj`, but nothing in the tests/ trampoline calls it (`tests/build/aarch32-luckfox-lyra-cmake-gcc-debug/build.ninja:705`). Consequence: on the Lyra the harness suite runs on core 0 only, and the suite's SMP assumptions (idle threads per core / migration) are never exercised. This is a divergence from the AArch32 rpi support that should be shared, not a local stylistic choice. Severity High (functional test-coverage hole on the only hardware-only AArch32 board); confirmed.

[High] `platform-support` / `base platform` are dead code on the four Raspberry Pi platforms — `tests/platforms/aarch32-rpi3b/cmake/platform-library.cmake:102`

`platform-library.cmake` builds `micro-os-plus::platform` and `micro-os-plus::platform-support` for `aarch32-rpi-zero-2w`, `aarch32-rpi3b`, `aarch64-rpi-zero-2w`, `aarch64-rpi3b`. None of them is referenced anywhere: the port's own builder compiles `src/platform-support.cpp` directly (`micro-os-plus-iii-aarch32.git/test/rpi3b/tests.cmake:92`) and links the port, not these targets. Confirmed:

`micro-os-plus::platform-support` / `platform-aarch32-rpi-zero-2w-support-interface` do not appear in `tests/build/aarch32-rpi-zero-2w-cmake-gcc-debug/build.ninja` at all. The ~120 lines per platform are unreachable and, worse, misleading (see next finding). Low direct risk, but the misleading "adds the startup hooks ... for the harness suites" comment (`:96`) is contradicted by the code path that actually builds the suites.

[Medium] Raspberry Pi base platform hard-codes the QEMU variant — `tests/`

`platforms/aarch32-rpi3b/cmake/platform-library.cmake:54`

Even though these platforms register both `-qemu` and `-hwd` tests, the base interface unconditionally defines `QEMU_BUILD` and selects the QEMU linker:

```
QEMU_BUILD
...
target_link_options (
  platform-aarch32-rpi3b-interface
  INTERFACE -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_QEMU}"
)
```

and `platform-support` inherits it (`:112-120`). Today the hwd path avoids this only because the port's builder is used instead (previous finding). If the dead interface is ever revived for a hardware image, it will silently select the emulator's SVC/reason-by-value semihosting and the QEMU linker script — exactly the trap the Luckfox file warns about in `tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake:16` . Same pattern in `aarch64-rpi3b:53,69` and both `rpi-zero-2w` files. Confirmed code; latent today.

[Medium] Pinned-toolchain guard exists only on aarch32/aarch64 — `tests/`

`platforms/aarch32-rpi3b/cmake/definitions.cmake:34`

The guard that refuses a non-15.2 compiler is present only in the aarch32/aarch64 definitions:

```
if (DEFINED CMAKE_C_COMPILER_VERSION
    AND NOT CMAKE_C_COMPILER_VERSION MATCHES "^15\\.2\\.")
  message (FATAL_ERROR "This platform must be built with the pinned xPack arm-none-eabi-gcc
  15.2 ...")
```

`qemu-cortex-*`, `cortexm-*`, `raspberrypi-pico`, `nucleo-*` and all `native-*` configs have no equivalent, even though they also pin tools in `package.json` (`tests/package.json:551` arm-none-eabi-gcc 15.2.1, `:659` qemu 8.2.6, gcc/clang versions). Combined with the fact that `commandCMakeReconfigure` omits `CMAKE_TOOLCHAIN_FILE` (it is only added by `commandCMakePrepareWithToolchain`, `tests/package.json:50-51`), a bare `cmake` or a build tree whose `xpack/` was removed silently falls back to `/usr/bin/cc` (the doc itself names this failure: `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:583`). Confirmed inconsistency; Medium.

[Medium] Legacy `else` fallback pairs the local SMP kernel with the released Cortex-M port — `tests/cmake/tests-main.cmake:106`

```
else ()
    # Fallback: the plain kernel, one level above (upstream behaviour).
    add_subdirectory (".." "top-bin")
endif ()
```

This branch is taken by `qemu-cortex-*`, `raspberrypi-pico` and `nucleo-*`. Their `dependencies-folders.cmake` then also add the *published* port, e.g. `tests/platforms/qemu-cortex-m4f/cmake/dependencies-folders.cmake:30`:

```
"${CMAKE_BINARY_DIR}/xpacks/@micro-os-plus/micro-os-plus-iii-cortexm"
```

i.e. `@micro-os-plus/micro-os-plus-iii-cortexm@1.1.0` from `package.json:525`, while the kernel under test is the *local* `..` (this SMP repo). The newer `cortexm-*` platforms instead `add_subdirectory ${UOS_CORTEXM_DIR}` (the local port). So the same harness tests two different kernel/port pairings depending on platform; the qemu/nucleo/raspberrypi ones test the local kernel against a released port, which is a stale combination and a reproducibility hazard. Confirmed; Medium.

[Medium] `ENABLE_HW_TESTS=ON` is passed but no CMake reads it — `tests/package.json:1725`

```
"commandCMakeReconfigure": "cmake ... -D PLATFORM_NAME={{ properties.platformName }} -D
ENABLE_HW_TESTS=ON"
```

`rg ENABLE_HW_TESTS` finds it only in docs (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:312`, `docs/tests/WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md:911`) and this `package.json`. The actual `tests/platforms/aarch32-luckfox-lyra/CMakeLists.txt` neither declares the option nor branches on it; it always registers every `hwd` case. So the flag is a no-op and the documented `option(ENABLE_HW_TESTS ...)` does not exist. Confirmed; Medium (dead/drifted configuration).

[Medium] AArch64 support stubs `_gettimeofday` to a constant — `tests/platforms/aarch64-rpi3b/src/platform-support.cpp:161`

```
int
_gettimeofday (struct timeval* tv, void*)
{
    if (tv != nullptr) { tv->tv_sec = 0; tv->tv_usec = 0; }
    return 0;
}
```

The suites exercise the realtime clock heavily (`tests/sources/rtos-apis/src/test-iso-api.cpp:461-511`, `sleep_for<realtime_clock>`, `wait_until(realtime_clock::now()+...)`). A constant epoch makes elapsed-time deltas zero, so those assertions can pass (or misbehave) for the wrong reason. The comment justifies the stub for mutex-stress, but on AArch64 `__ARM_EABI__` is defined, so mutex-stress takes the `hrclock` branch, not `gettimeofday`. Suspicion (Medium) — I did not run an AArch64 image to prove a false pass; worth a targeted check.

[Medium] `bash`-only CTest commands undermine native-Windows support —

`tests/platforms/native/CMakeLists.txt:52`

```
COMMAND bash "${_runner}/run-host.sh" "${_port_bin}"
```

The new harness registers `bash run-*.sh` for native, qemu and hwd everywhere (`tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt:73,87`, `tests/platforms/cortexm-pico2/CMakeLists.txt:86`, ...). The legacy ports at least tried a Windows shim (`tests/cmake/tests-main.cmake:35-37` sets `extension = .cmd` and legacy commands use `qemu-system-arm${extension} / openocd${extension}`). `tests/README.md:39` claims the tests run on Windows. Unless a Git-Bash `bash` is guaranteed on `PATH`, the entire new harness is non-runnable on native Windows; and `ENVIRONMENT "...;..."` with semicolon-separated absolute Windows paths (`aarch32-rpi-zero-2w/CMakeLists.txt:79-80`) is fragile for paths with spaces. Confirmed code; portability suspicion (Medium).

[Medium] Freshness/labels/timeout missing on the legacy QEMU platforms —

`tests/platforms/qemu-cortex-m4f/CMakeLists.txt:61`

```
add_test (
  NAME "rtos-apis-test"
  COMMAND qemu-system-arm${extension} --machine mps2-an386 ...
)
```

Unlike the newer platforms, these tests carry no `LABELS`, no `TIMEOUT`, and names without the `_${PLATFORM_NAME}-` prefix (the project's own rule, `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:567`). A hung emulator therefore burns CTest's 1500 s default, and there is no `qemu` label to filter. Same for `qemu-cortex-m0/m3/m7f` and the legacy `nucleo-*` / `raspberrypi-pico` names. Confirmed; Medium.

[Medium] Two Cortex-M SMP-QEMU platforms run QEMU with no log capture —

`tests/platforms/2xcortex-m33/CMakeLists.txt:37`

```
add_test (
  NAME "${PLATFORM_NAME}-${name}"
  COMMAND "${_qemu}" --machine ${_qemu_machine} --cpu ${_qemu_cpu} --smp 2 ...
)
```

`2xcortex-m33` and `pico2-1cpu` (and the `cortexm-pico2` qemu cases) invoke QEMU directly, whereas the AArch ports go through `test_smpl/run-qemu.sh`, which tees each run to `.qemu-logs/<app>.log` (`test_smpl/run-qemu.sh:95,118`). `tests/build/2xcortex-m33-cmake-gcc-debug/Testing/Temporary/LastTestsFailed.log` records `2xcortex-m33-fp-switch-test` failing at some point; a re-run of the exact existing image passed (79 migrations, 0 bad), but because no log was kept the original failure cannot be diagnosed. Robustness/diagnostics gap; Medium.

[Medium] Hard-coded sibling-repo paths with a `.git` suffix — `tests/cmake/tests-`

`main.cmake:62`

```
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git"
    CACHE PATH "µOS++ III AArch32 port working copy")
```

The harness only works if the sibling checkouts are literally named `*.git`; the project's own cheat-sheet documents the workaround of symlinking them (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:623-625`). The values are `CACHE PATH`, so a stale `CMakeCache.txt` keeps pointing at a previous path even after the tree moves. Confirmed; Medium (build-config hygiene).

[Medium] Dead validation messages point at the wrong file — `tests/device-qemu-`

`cortexm/CMakeLists.txt:44`

```
message (FATAL_ERROR
    "Define xpack_device_linker_script_file_name in platforms/${PLATFORM_NAME}/cmake/dependencies-
    folders.cmake")
```

The variable is actually set in `cmake/definitions.cmake` (e.g. `tests/platforms/qemu-cortex-m4f/cmake/definitions.cmake:29`), as is `xpack_platform_compile_definition` referenced by `tests/platforms/qemu-cortex-m4f/cmake/platform-library.cmake:30`. This message is reproduced in every generated platform library. Low severity but actively misleading during bring-up. Confirmed.

[Low] `run-hw.sh` passes a user-controlled string to `printf` as format — `test_smpl/`

`run-hw.sh:93`

```
target_name () { printf "$TARGET_FMT" "$1"; }
```

`UOS_HW_TARGET_FMT` comes from the board (default `bcm2837.cpu%d`). ShellCheck flags SC2059; a stray `%` in a future board config produces garbage or a printf error rather than a clean failure. Also the whole OpenOCD command is assembled by string interpolation of `UOS_HW_*` into a `.cfg` (`:203-216`), which is fine for trusted board files but is a quoting/injection foot-gun. Low; Nit.

[Low] `native` runner scan and exit-on-first-non-executable — `test_smpl/run-`

`host.sh:57`

```
for exe in "${BUILD_DIR}"/*-host; do
  [[ -x "$exe" ]] || { echo "no *-host executables in ${BUILD_DIR}"; exit 2; }
```

Each of the ~17 `native-<app>-host` CTest cases runs `run-host.sh`, which scans the whole directory and filters by `UOS_RUN_ONLY` (`tests/platforms/native/CMakeLists.txt:50-57`). Besides the $O(n^2)$ work, the `-x` check runs before the filter, so a single non-executable `*-host` entry makes every case exit 2. Low.

[Low] Legacy QEMU README errors — `tests/platforms/qemu-cortex-m0/README.md:35`

```
qemu-system-arm --machine mps2-an365 --cpu cortex-m3 ...
qemu-system-arm --machine mps2-an5365 ...
```

`an365` / `an5365` are typos for `an385` (the CMake uses `mps2-an385`, `tests/platforms/qemu-cortex-m0/CMakeLists.txt:68`). `tests/platforms/qemu-cortex-m3/README.md:1` is titled `# platforms/qemu-cortex-m0`. The M0-on-M3 CPU choice itself is intentional and documented in `docs/tests/TESTS-CATALOG.md:65`, so only the typos/title are defects. Low.

[Low] `TRACE` / `-v` leak from the shared interface — `tests/cmake/common-options.cmake:32`

```
$$<CONFIG:Debug>:DEBUG>
$$<CONFIG:Debug>:TRACE>
OS_USE_OS_APP_CONFIG_H
```

`micro-os-plus::common-options` is linked PRIVATE by every executable, so a compiled-in `TRACE` and the linker `-v` (`:64`) apply to *all* sources of every suite, including suites that did not opt in. The old `mutex-stress` header warns that trace in the scheduler interrupt "occasionally" breaks tests (`tests/README.md:74-75`). Intentional for Debug perhaps, but the interface name promises "common" while injecting a debug-only diagnostic macro. Suspicion/Low.

[Low] `EXCLUDE_FROM_ALL` on INTERFACE libraries is meaningless —

`tests/sources/rtos-apis/CMakeLists.txt:30`

```
add_library (test-rtos-apis-interface INTERFACE EXCLUDE_FROM_ALL)
```

`EXCLUDE_FROM_ALL` has no effect on INTERFACE libraries (they produce no build artifacts). Same in every `sources/*/CMakeLists.txt` and `tests/cmake/common-options.cmake:25`. Nit — but it suggests the pattern was copied from object libraries.

[Low] `ENABLE_*` globals only affect legacy platforms — `tests/cmake/global-definitions.cmake:16`

```
set (ENABLE_RTOS_APIS_TEST true)
set (ENABLE_MUTEX_STRESS_TEST true)
set (ENABLE_CMSIS_OS_VALIDATOR_TEST true)
```

Only `qemu-cortex-*`, `nucleo-*` and `raspberrypi-pico` consult these. `native` builds `cmsis-os-validator-test` unconditionally and ignores the flags (`tests/platforms/native/CMakeLists.txt:70`), and the `aarch32/aarch64/cortexm` platforms build everything the port's builder emits. Low (dead configuration surface).

[Low] `package.json` link/install hygiene — `tests/package.json:157`

`link-deps-all` repeats `native-cmake-gcc13-debug` and omits a `gcc14-release` pairing:

```
"xpm run link-deps --config native-cmake-gcc13-debug",
"xpm run link-deps --config native-cmake-gcc13-release",
"xpm run link-deps --config native-cmake-gcc13-debug",
"xpm run link-deps --config native-cmake-gcc14-release",
```

Separately, the native dependency key is `@micro-os-plus/posix-arch` (`:509`) but the link action uses `@micro-os-plus/micro-os-plus-iii-posix-arch` (`:489`); `xpm` happens to install under the package's real name (`tests/build/native-cmake-gcc-debug/xpacks/@micro-os-plus/micro-os-plus-iii-posix-arch`), so the key is misleading. Low.

[Low] Platform-support is quadruplicated and already divergent — `tests/platforms/aarch32-rpi3b/src/platform-support.cpp:1`

The two AArch32 files differ only in comments (verified by `diff`), and the two AArch64 files likewise; AArch32 vs AArch64 differ in the C-library policy. The root repo already offers `micro-os-plus::test-support` for shared scaffolding, but it only carries `hw_result.hpp`/`board-contract.cpp` (`CMakeLists.txt:226-230`), so the ~150-line startup/`main`/console-mirror body is copied four times — which is precisely how the Luckfox file lost its SMP bring-up. Maintainability; Low. Suggest a shared header/target so the divergence cannot recur.

[Low] Stale `tests/README.md` — `tests/README.md:56`

```
For Cortex-M tests, the toolchain is arm-none-eabi-gcc 14.
```

The pin is 15.2.1 (`tests/package.json:551`). The file also references `micro-os-plus-iii.git` throughout (`:64,69`), `.github/workflows/{ci,test-all}.yaml` that do not exist in this repo, and omits the newer platforms (`aarch32-*`, `aarch64-*`, `cortexm-pico2`, `2xcortex-m33`, `pico2-1cpu`,

`cortexm-*`). Unlike the `docs/tests/` harness documents it has **no** "earlier layout — read as history" banner, so it reads as current. Medium-adjacent, but doc-only → Low.

[Low] Developer/maintainer READMEs point at the wrong repo/version — `docs/`

`tests/README-DEVELOPER.md:53`

```
xpm run install-native-cmake-sys -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install-native-cmake-sys -C ~/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests
```

The second command is a copy-paste of the first (should be `test-native-cmake-sys` or `test`). Every path uses the old `micro-os-plus-iii.git` repo. `docs/tests/README-MAINTAINER.md:86-93,116` references a `CHANGELOG.md` and version `7.1.0` that do not exist here (`tests/package.json` is `0.0.0`; no `CHANGELOG.md`). Low.

[Low] Naming rule violated by legacy platforms — `tests/platforms/qemu-cortex-m0/`

`CMakeLists.txt:65`

The docs state the rule "Unique test names + project name per platform (`${PLATFORM_NAME}-...`)" (`docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md:567`), but `qemu-cortex-*` register bare `rtos-apis-test`/`mutex-stress-test`/`cmsis-os-validator-test`, and `raspberrypi-pico`/`nucleo-*` do the same. Collisions are avoided only because each configuration is a separate build tree; a combined `ctest` across trees would collide. Low.

Documentation drift

- `tests/README.md` — genuinely stale (no history banner): pins "arm-none-eabi-gcc 14" vs `package.json:551` = 15.2.1; references `micro-os-plus-iii.git` and nonexistent `.github/workflows/*`; platform/toolchain list predates aarch32/aarch64/cortexm-pico2/2xcortex/pico2-1cpu.
- `docs/tests/README-DEVELOPER.md` — duplicated `install-native-cmake-sys` (`:51,53`), all paths under the old repo name.
- `docs/tests/README-MAINTAINER.md` — references `CHANGELOG.md` and release `7.1.0` that do not exist here; old repo path.
- `docs/tests/HARNESS-BOARD-TEST-CHEATSHEET.md` / `HARNESS-TESTS-PARADIGM.md` — carry an explicit "earlier layout — read as history" banner at the top, so the wrong `micro-os-plus-iii.git` paths and the sample ctest output `aarch32-rpi-zero-2w-mutex-stress-test` (actual names are `...-mutex-stress-qemu`/`-hwd`, `tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt:71,86`) are self-declared historical, **except**: `HARNESS-TESTS-PARADIGM.md:228` still recommends `xpm run test-mutex-stress --config ...`, an action that does not exist (package.json only defines `test-mutex-stress-qemu`/`-hwd`), and the "which library is under test" table (`:176-181`) omits the `native`/`cortexm`/`pico2`/`2xcortex` branches that `tests-main.cmake:84-105` now has. Consider moving the current truth into `docs/tests/STEPS.md`/`TESTS-CATALOG.md` (which are current) and deleting or clearly archiving the rest.

- `ENABLE_HW_TESTS` — documented as an option in the cheat-sheet/guide but nonexistent in the CMake (see finding).

Good practices

- The "platform registers what the port's builder produced by `file(GLOB)` -ing `test/<board>/*`" seam means a test added to a port appears with no harness edit (`tests/platforms/aarch32-rpi3b/CMakeLists.txt:63-110`), and the tests-main comment at `:55-60` explicitly documents the duplicate-`micro-os-plus::iii` -alias hazard.
- `run-qemu.sh` / `run-host.sh` / `run-hw.sh` share a single verdict protocol (`RESULT: PASS/SKIP/FAIL` , `TIMEOUT` , `NO-RESULT`) and per-test wall-clock tables, with clear separation of the emulator and the probe; `run-hw.sh` correctly refuses to run a suite (`:280-286`) and reports a lost DAP distinctly from a firmware fault.
- Platform-support mirrors the port's own startup hooks exactly and explains why `putc_uart()` rather than `putc()` is used for the UART mirror (`tests/platforms/aarch32-rpi3b/src/platform-support.cpp:176-179`).
- The `aarch32-luckfox-lyra` platform deliberately omits `QEMU_BUILD` and documents why selecting the emulator trap under OpenOCD would report the wrong status — a good example of encoding a non-obvious board fact next to the code.
- `tests/cmake/tests-main.cmake:35-37` centralizes the Windows command-extension shim, and `package.json` defines per-config tool dependencies rather than relying on the host.